

## Appendix E. C Idioms in Assembly Language

This appendix is for C programmers learning assembly language. It is meant to give a general idea about how C constructs can be implemented in assembly language.

### If Statement

In C, an if statement consists of three parts - the condition, the true branch, and the false branch. However, since assembly language is not a block structured language, you have to work a little to implement the block-like nature of C. For example, look at the following C code:

```
if(a == b)
{
    /* True Branch Code Here */
}
else
{
    /* False Branch Code Here */
}

/* At This Point, Reconverge */
```

In assembly language, this can be rendered as:

```
#Move a and b into registers for comparison
movl a, %eax
movl b, %ebx

#Compare
cmpl %eax, %ebx

#If True, go to true branch
je true_branch
```

## *Appendix E. C Idioms in Assembly Language*

```
false_branch:  #This label is unnecessary,
               #only here for documentation
               #False Branch Code Here
```

```
               #Jump to reconvergence point
               jmp reconverge
```

```
true_branch:
               #True Branch Code Here
```

```
reconverge:
               #Both branches reconverge to this point
```

As you can see, since assembly language is linear, the blocks have to jump around each other. Reconvergence is handled by the programmer, not the system.

A case statement is written just like a sequence of if statements.

### **Function Call**

A function call in assembly language simply requires pushing the arguments to the function onto the stack in *reverse* order, and issuing a `call` instruction. After calling, the arguments are then popped back off of the stack. For example, consider the C code:

```
printf("The number is %d", 88);
```

In assembly language, this would be rendered as:

```
.section .data
text_string:
.ascii "The number is %d\0"
```

```
.section .text
pushl $88
pushl $text_string
call  printf
popl  %eax
popl  %eax      #%eax is just a dummy variable,
                #nothing is actually being done
                #with the value.  You can also
                #directly re-adjust %esp to the
                #proper location.
```

## **Variables and Assignment**

Global and static variables are declared using `.data` or `.bss` entries. Local variables are declared by reserving space on the stack at the beginning of the function. This space is given back at the end of the function.

Interestingly, global variables are accessed differently than local variables in assembly language. Global variables are accessed using direct addressing, while local variables are accessed using base pointer addressing. For example, consider the following C code:

```
int my_global_var;

int foo()
{
    int my_local_var;

    my_local_var = 1;
    my_global_var = 2;

    return 0;
}
```

## Appendix E. C Idioms in Assembly Language

This would be rendered in assembly language as:

```
.section .data
.lcomm my_global_var, 4

.type foo, @function
foo:
    pushl %ebp           #Save old base pointer
    movl %esp, %ebp      #make stack pointer base pointer
    subl $4, %esp        #Make room for my_local_var
    .equ my_local_var, -4 #Can now use my_local_var to
                        #find the local variable

    movl $1, my_local_var(%ebp)
    movl $2, my_global_var

    movl %ebp, %esp      #Clean up function and return
    popl %ebp
    ret
```

What may not be obvious is that accessing the global variable takes fewer machine cycles than accessing the global variable. However, that may not matter because the stack is more likely to be in physical memory (instead of swap) than the global variable is.

Also note that in the C programming language, after the compiler loads a value into a register, that value will likely stay in that register until that register is needed for something else. It may also move registers. For example, if you have a variable `foo`, it may start on the stack, but the compiler will eventually move it into registers for processing. If there aren't many variables in use, the value may simply stay in the register until it is needed again. Otherwise, when that register is needed for something else, the value, if it's changed, is copied back to its corresponding memory location. In C, you can use the keyword `volatile` to make sure all modifications and references to the variable are done to the memory

location itself, rather than a register copy of it, in case other processes, threads, or hardware may be modifying the value while your function is running.

## **Loops**

Loops work a lot like if statements in assembly language - the blocks are formed by jumping around. In C, a while loop consists of a loop body, and a test to determine whether or not it is time to exit the loop. A for loop is exactly the same, with optional initialization and counter-increment sections. These can simply be moved around to make a while loop.

In C, a while loop looks like this:

```
while(a < b)
{
    /* Do stuff here */
}

/* Finished Looping */
```

This can be rendered in assembly language like this:

```
loop_begin:
    movl  a, %eax
    movl  b, %ebx
    cmpl  %eax, %ebx
    jge   loop_end

loop_body:
    #Do stuff here

    jmp  loop_begin

loop_end:
    #Finished looping
```

## Appendix E. C Idioms in Assembly Language

The x86 assembly language has some direct support for looping as well. The `%ecx` register can be used as a counter that *ends* with zero. The `loop` instruction will decrement `%ecx` and jump to a specified address unless `%ecx` is zero. For example, if you wanted to execute a statement 100 times, you would do this in C:

```
for(i=0; i < 100; i++)
{
    /* Do process here */
}
```

In assembly language it would be written like this:

```
loop_initialize:
    movl $100, %ecx
loop_begin:
    #
    #Do Process Here
    #

    #Decrement %ecx and loops if not zero
    loop loop_begin

rest_of_program:
    #Continues on to here
```

One thing to notice is that the `loop` instruction *requires you to be counting backwards to zero*. If you need to count forwards or use another ending number, you should use the `loop` form which does not include the `loop` instruction.

For really tight loops of character string operations, there is also the `rep` instruction, but we will leave learning about that as an exercise to the reader.

## Structs

Structs are simply descriptions of memory blocks. For example, in C you can say:

```
struct person {  
    char firstname[40];  
    char lastname[40];  
    int age;  
};
```

This doesn't do anything by itself, except give you ways of intelligently using 84 bytes of data. You can do basically the same thing using `.equ` directives in assembly language. Like this:

```
.equ PERSON_SIZE, 84  
.equ PERSON_FIRSTNAME_OFFSET, 0  
.equ PERSON_LASTNAME_OFFSET, 40  
.equ PERSON_AGE_OFFSET, 80
```

When you declare a variable of this type, all you are doing is reserving 84 bytes of space. So, if you have this in C:

```
void foo()  
{  
    struct person p;  
  
    /* Do stuff here */  
}
```

In assembly language you would have:

```
foo:  
    #Standard header beginning  
    pushl %ebp  
    movl %esp, %ebp  
  
    #Reserve our local variable  
    subl $PERSON_SIZE, %esp  
    #This is the variable's offset from %ebp
```

## *Appendix E. C Idioms in Assembly Language*

```
.equ P_VAR, 0 - PERSON_SIZE
```

```
#Do Stuff Here
```

```
#Standard function ending
```

```
movl %ebp, %esp
```

```
popl %ebp
```

```
ret
```

To access structure members, you just have to use base pointer addressing with the offsets defined above. For example, in C you could set the person's age like this:

```
p.age = 30;
```

In assembly language it would look like this:

```
movl $30, P_VAR + PERSON_AGE_OFFSET(%ebp)
```

## **Pointers**

Pointers are very easy. Remember, pointers are simply the address that a value resides at. Let's start by taking a look at global variables. For example:

```
int global_data = 30;
```

In assembly language, this would be:

```
.section .data
```

```
global_data:
```

```
.long 30
```

Taking the address of this data in C:

```
a = &global_data;
```

Taking the address of this data in assembly language:

```
movl $global_data, %eax
```

You see, with assembly language, you are almost always accessing memory through pointers. That's what direct addressing is. To get the pointer itself, you just have to go with immediate mode addressing.

Local variables are a little more difficult, but not much. Here is how you take the address of a local variable in C:

```
void foo()  
{  
    int a;  
    int *b;  
  
    a = 30;  
  
    b = &a;  
  
    *b = 44;  
}
```

The same code in assembly language:

```
foo:  
    #Standard opening  
    pushl %ebp  
    movl  %esp, %ebp  
  
    #Reserve two words of memory  
    subl  $8, $esp  
    .equ  A_VAR, -4  
    .equ  B_VAR, -8  
  
    #a = 30
```

## *Appendix E. C Idioms in Assembly Language*

```
movl $30, A_VAR(%ebp)

#b = &a
movl $A_VAR, B_VAR(%ebp)
addl %ebp, B_VAR(%ebp)

#*b = 30
movl B_VAR(%ebp), %eax
movl $30, (%eax)

#Standard closing
movl %ebp, %esp
popl %ebp
ret
```

As you can see, to take the address of a local variable, the address has to be computed the same way the computer computes the addresses in base pointer addressing. There is an easier way - the processor provides the instruction `leal`, which stands for "load effective address". This lets the computer compute the address, and then load it wherever you want. So, we could just say:

```
#b = &a
leal A_VAR(%ebp), %eax
movl %eax, B_VAR(%ebp)
```

It's the same number of lines, but a little cleaner. Then, to use this value, you simply have to move it to a general-purpose register and use indirect addressing, as shown in the example above.

## **Getting GCC to Help**

One of the nice things about GCC is it's ability to spit out assembly language code. To convert a C language file to assembly, you can simply do:

```
gcc -S file.c
```

The output will be in `file.s`. It's not the most readable output - most of the variable names have been removed and replaced either with numeric stack locations or references to automatically-generated labels. To start with, you probably want to turn off optimizations with `-O0` so that the assembly language output will follow your source code better.

Something else you might notice is that GCC reserves more stack space for local variables than we do, and then AND's `%esp`<sup>1</sup> This is to increase memory and cache efficiency by double-word aligning variables.

Finally, at the end of functions, we usually do the following instructions to clean up the stack before issuing a `ret` instruction:

```
movl %ebp, %esp
popl %ebp
```

However, GCC output will usually just include the instruction `leave`. This instruction is simply the combination of the above two instructions. We do not use `leave` in this text because we want to be clear about exactly what is happening at the processor level.

I encourage you to take a C program you have written and compile it to assembly language and trace the logic. Then, add in optimizations and try again. See how the compiler chose to rearrange your program to be more optimized, and try to figure out why it chose the arrangement and instructions it did.

---

1. Note that different versions of GCC do this differently.

